Uma Breve Introdução a Banco de Dados

A área de banco de dados é de grande importância no mundo da informática, uma vez que a informação é um bem precioso e deve ser armazenada de forma coerente e adequada.

Quando você utiliza o Orkut, por exemplo, armazena em um sistema de banco de dados suas informações pessoais, recados de amigos, depoimentos, etc. Se você sair da sua página no Orkut e entrar nela novamente, as informações ainda estarão lá. Isso ocorre porque essas informações foram armazenadas em um banco de dados, e, portanto, podem ser recuperadas no momento oportuno, quando o usuário solicitar.

Atualmente, por menor e mais simples que seja um Sistema de Informação, ele precisará ter a capacidade de armazenar e recuperar dados rapidamente. Por exemplo, se você desenvolver um Sistema de Informação para um salão de beleza do seu bairro, este sistema terá que armazenar dados de clientes, produtos, valores, funcionários, serviços, etc. É para armazenar essas informações e recuperá-las rapidamente que utilizamos um sistema de banco de dados.

No entanto, para que possamos entender o que está compreendido na área de banco de dados, ou seja, quais os modelos, quais as ferramentas disponíveis, como modelar, implementar uma base de dados e como recuperar dados, é necessário compreender alguns conceitos básicos sobre o assunto.

O Que São Dados? O Que É Informação?

Dados são tudo que podemos inferir ou coletar sobre uma situação específica. Os dados podem ser úteis ou não. Por exemplo, em uma sala de aula a quantidade de carteiras, a cor da parede, o tipo do assoalho, as dimensões da sala, etc., fornecem-nos dados sobre o ambiente. No entanto, no desenvolvimento de uma aplicação, esses dados podem ser úteis ou não, dependendo do objetivo do projeto. Se, por exemplo, uma arquiteta tem a intenção de desenvolver um novo leiaute para aquela sala de aula, os dados acima mencionados serão úteis.

Os dados úteis é o que chamamos de **informação**. E esses dados são o que armazenamos em uma base de dados.

Por convenção, na área de banco de dados, os termos “informação” e “dado” significam a mesma coisa. Isso ocorre porque devemos armazenar apenas aquilo que é útil para a nossa aplicação. Sendo assim, daqui para frente, os dois termos serão usados como sinônimos.

O Que É uma Base de Dados?

Uma base de dados é um local, ou espaço, onde informações estão armazenadas e de onde elas são recuperadas. Uma base de dados terá um nome, e este nome deverá representar o que aquela base armazena. Por exemplo, se a aplicação for uma agenda de contatos pessoal e profissional, o nome da base poderá ser bd_Agenda. Essa base de dados deverá armazenar todas as informações sobre os contatos pessoais e profissionais como, por exemplo, nome, endereço, tipo do endereço (residencial, comercial, de referência, etc.), telefone residencial, telefone celular, telefone comercial, nome da empresa em que trabalha, grau de parentesco, e-mail, etc.

Uma base de dados permite que os dados fiquem centralizados e que se relacionem de forma coerente.

O Que É um Sistema de Banco de Dados?

Computacionalmente, um Sistema de Banco de Dados é uma ferramenta que será utilizada para armazenar informações. Essa ferramenta possui três principais características:

- Armazenar os dados.
- Relacionar os dados armazenados.
- Recuperar os dados rapidamente.

Armazenar os dados significa que a ferramenta possui um repositório onde as informações são gravadas. Esse repositório permite centralizar os dados, evitando que eles fiquem espalhados em vários arquivos.

Relacionar os dados armazenados é muito importante. Imagine uma base de dados em uma escola que contenha informações sobre os alunos, os professores, as disciplinas, as turmas e os cursos. Se não for possível relacionar essas informações, como se saberá que um determinado aluno faz o curso de Técnico em Informática e tem aula da disciplina de Banco de Dados com determinado professor? Portanto, é importante que um sistema de Banco de Dados permita relacionar as informações armazenadas de forma coerente.

Lembre-se!

Armazenar dados e não relacioná-los não é nada interessante para quem desenvolve um Sistema de Informação.

Recuperar os dados por meio de consultas ao Sistema de Banco de Dados. Nos bancos de dados relacionais, as consultas são feitas utilizando-se a Linguagem **SQL**, cuja apresentação será feita nos capítulos 7, 8, 9 e 10.

***SQL* (Strutured Query Language):** *Linguagem de consulta estruturada.*

Cabe dizer que existem outros tipos de Sistemas de Banco de Dados além do relacional, embora esse seja o mais utilizado no mercado. Procure se informar sobre quais são os outros tipos de banco de dados.

O Que É um Sistema Gerenciador de Banco de Dados (SGBD)?



Exemplos de SGBD utilizados atualmente:
- Oracle Database (da Oracle)
- SQL Server (da Microsoft)
- PostgreSQL (código aberto)
- DB2 (da IBM)
- MySQL (código aberto, atualmente da Oracle)
- MariaDB (código aberto)

Atualmente, os Sistemas de Banco de Dados evoluíram para **Sistemas Gerenciadores de Banco de Dados (SGBD)**.

Um SGBD é uma ferramenta muito mais completa que um Sistema de Banco de Dados. Um SGBD disponibiliza uma série de funcionalidades que permitem controlar e acompanhar melhor os dados armazenados.

As principais características de um SGBD são:

- Permitir o acesso concorrente às bases de dados;
- Realizar o gerenciamento de transações;
- Permitir criar e aplicar regras de segurança às bases de dados;
- Permitir criar regras que garantam a integridade da base de dados.

Acesso Concorrente

O acesso concorrente à base de dados significa que o SGBD permite que duas ou mais pessoas acessem uma mesma base de dados ao mesmo tempo e o sistema controla para que um acesso não interfira no outro.

Por exemplo, em um sistema de compras na Web, várias pessoas podem realizar uma compra ao mesmo tempo, e o próprio SGBD controla para que os dados de todas as compras sejam gravados corretamente na base de dados.

Gerenciamento de Transações

Uma transação em banco de dados consiste em um conjunto de operações que é tratado como uma **unidade lógica indivisível**. Isso significa que quando começa a execução de uma transação, esta deve ter executadas **todas** as operações dentro dela.

Se acontecer qualquer falha durante a execução da transação (por exemplo: falta de energia, alguém desligar o servidor, cancelamento da transação pelo usuário, etc.) as operações pendentes devem ser canceladas, e aquelas que foram executadas deverão ser desfeitas. Isso acontece para garantir a integridade dos dados dentro da base.

Por exemplo, imagine que você está fazendo a compra de um jogo de computador pela Internet. Suponha que você já escolheu o jogo, colocou no carrinho de compras, preencheu o cadastro e entrou com os dados do seu cartão de crédito. Agora só falta você confirmar a compra.

No entanto, antes de você confirmar a compra ocorre um erro (que pode ser na sua máquina ou no servidor que tem a aplicação). Nessa situação, como a compra não foi confirmada, ela é cancelada, e todas as operações que haviam sido executadas anteriormente são desfeitas. Isso significa que o jogo que você estava tentando obter se tornará disponível para compra novamente, e poderá ser adquirido por outra pessoa.

Ainda sobre as transações: imagine que você queira trocar os pontos do seu cartão fidelidade por algum produto do seu interesse. Durante a troca (que é feita via sistema pela Internet ou na loja) ocorre uma falha. Você já havia escolhido o produto e o sistema já havia verificado que seus pontos eram suficientes para obtê-lo, e também já havia lhe dito quantos pontos ainda sobrariam no seu cartão. Como a falha ocorreu antes da troca ser confirmada, os pontos do seu cartão não poderão ser debitados e a troca será cancelada.



Um questionamento interessante:

E se a falha ocorresse após a confirmação da troca, mas antes da emissão do comprovante?

Nesse caso, mesmo que o problema seja no servidor, se a transação foi concluída antes da falha, o SGBD tem um mecanismo que verifica que essa transação foi concluída com sucesso e quando o servidor voltar a funcionar o SGBD fará as atualizações necessárias na base de dados. Assim, os pontos serão debitados e você receberá o seu produto em troca dos pontos.

Regras de Segurança

Outro aspecto importante com o qual o SGBD se preocupa é com a segurança das informações armazenadas. Sendo assim, um SGBD tem mecanismos para criação de regras de segurança.

As regras de segurança vão desde a definição de *login* e senha para os usuários, até a permissão de acesso ao SGBD e acesso aos dados armazenados.

Em relação às regras de permissão de acesso ao SGBD, é possível definir o que o usuário pode fazer no SGBD, ou seja, definir o papel do usuário no SGBD. Por exemplo: posso ter um usuário do SGBD que só tem permissão para leitura de dados; ou outro usuário que tenha permissão para criar base de dados e manipulá-la, mas não pode criar novos usuários ou realizar *backup*.

Quanto ao acesso aos dados, pode-se definir em uma base de dados qual usuário tem acesso a qual informação. Por exemplo: pode-se definir que os alunos tenham acesso aos dados de disciplinas e turmas, mas que não possam acessar dados do professor.

Regras de Integridade

Outra característica importante de um SGBD a se destacar é a possibilidade de criação de regras que garantam a integridade da base de dados.

As regras de integridade são interessantes porque ficam definidas para uma base de dados, e todas as aplicações que acessaram aquela base poderão utilizar a regra de integridade.

Imagine a necessidade de desenvolver uma aplicação onde o usuário deve preencher o estado em que mora. Suponha que os valores permitidos são apenas "PR", "SC" e "RS". Pode-se escrever uma regra na base de dados que verifique se o estado é válido ou não. O próprio SGBD vai fazer a verificação todas as vezes que o estado for inserido na base de dados.

Outro exemplo: Supondo que em um sistema de compras pela Internet o usuário só possa parcelar suas compras se o valor da compra for superior a R$ 150,00. Pode-se criar essa regra no banco, e assim o próprio SGBD poderá validar isso para o usuário.

Além das quatro características citadas e explicadas acima, os SGBD possuem várias outras funcionalidades como, por exemplo, algoritmos para extração de conhecimento em base de dados (*data mining*), técnicas OLAP e de definição de *data warehouse*, etc.

Atualmente, os SGBD funcionam em uma arquitetura cliente-servidor. Isso significa que você pode instalar o SGBD em um servidor e instalar o cliente em várias máquinas para que vários usuários tenham acesso simultâneo ao SGBD. É importante ressaltar que nessa arquitetura todas as bases de dados são criadas, alteradas e excluídas no servidor e não na máquina do cliente.

Quem Usa um Banco de Dados?

Os usuários de um banco de dados podem ser divididos em 3 categorias:

1. **Administrador do banco de dados (DBA):** é o responsável por monitorar e gerenciar todas as bases de dados criadas no SGBD. Também é quem controla as permissões dos usuários, garante que os usuários tenham acesso aos dados, realiza *backups*, recupera os dados em caso de falhas, garante o melhor desempenho para o banco de dados, monitora serviços (*Jobs*) de usuários no banco de dados, etc. Normalmente, um DBA é responsável por um SGBD específico, e deve ter estudado e feito cursos para aquele SGBD (por exemplo: DBA Oracle ou DBA SQL Server).
2. **Analistas de sistemas e programadores de aplicações**: são responsáveis por modelar a base de dados e implementá-la no SGBD escolhido. Também são responsáveis por desenvolver a aplicação (programa escrito em uma linguagem de programação como: Java, PHP, C++, C#, etc.) e conectar essa aplicação à base de dados do sistema. Esse usuário precisa conhecer a área de banco de dados, saber modelar uma base de dados e também conhecer a linguagem SQL.
3. **Usuários finais**: os usuários finais são aquelas pessoas que vão trabalhar diariamente com as aplicações desenvolvidas. São eles os responsáveis pela entrada de dados no banco de dados e pelas alterações nos dados armazenados. Esses usuários não precisam ter nenhum conhecimento sobre banco de dados ou saber qual o SGBD utilizado. Para eles, o banco de dados é transparente, e só interessa que as informações estejam sendo salvas e possam ser recuperadas.

O esquema a seguir ilustra os papéis de cada usuário em uma hierarquia em camadas.

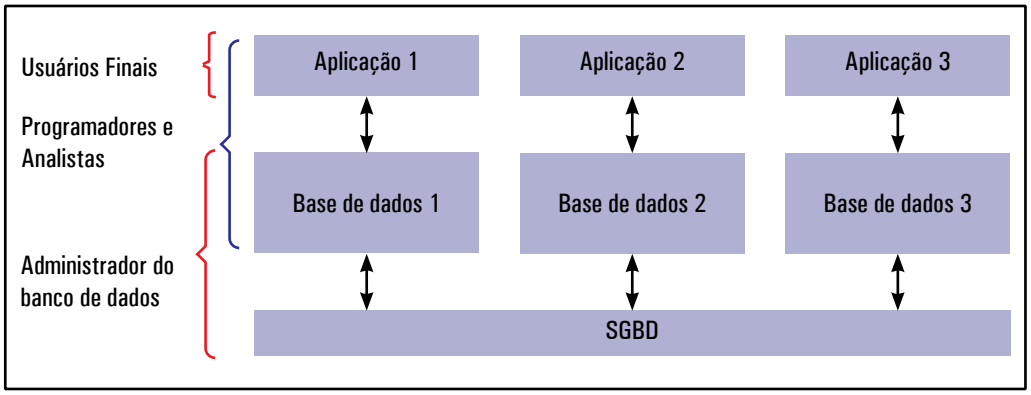


Figura 1.1 – Acesso dos usuários ao banco de dados

Os administradores de banco de dados devem manter o SGBD e monitorar as bases criadas naquele SGBD. Os analistas e programadores são responsáveis tanto pelo desenvolvimento das aplicações quanto pela modelagem e implementação da base de dados. Além disso, eles devem conectar a aplicação à base de dados, para que seja possível armazenar e recuperar os dados. Os usuários finais têm acesso apenas à aplicação e toda inserção, alteração, exclusão e consultas aos dados são feitas via aplicação.

Fases no Desenvolvimento de um Projeto de Banco de Dados

Um projeto de banco de dados compreende as seguintes fases:

- **Modelagem Conceitual:** A modelagem conceitual refere-se ao desenvolvimento de um modelo inicial da base de dados que reflita as necessidades do usuário. Essa modelagem preocupa-se em descrever **quais dados** serão armazenados na base de dados e **quais dados** se relacionam. Para fazer o modelo conceitual, é necessário entender o que o usuário final espera que o sistema armazene e que informações este usuário espera que o sistema disponibilize (como por exemplo, relatórios). Para obter as informações necessárias para desenvolver a modelagem conceitual do sistema, deve-se realizar entrevistas com o usuário para entender os objetivos do sistema e as expectativas que o usuário tem em relação a ele. Um dos principais modelos desta etapa é o *Modelo de Entidade e Relacionamento*.
- **Modelagem Lógica:** A modelagem lógica compreende o processo de descrever **como** os dados serão armazenados no sistema e **como** irão se relacionar. Isso significa transformar o modelo conceitual obtido na primeira fase num modelo mais próximo da implementação.

Para banco de dados relacionais, o modelo utilizado nessa fase é o modelo relacional. Também é necessário descrever o dicionário de dados da base de dados nessa etapa. Antes da fase de implementação, é necessário, ainda verificar se o modelo está normalizado e em caso negativo, deve-se normalizar o modelo.

- **Implementação do Modelo Lógico:** Uma vez que toda a etapa de modelagem esteja concluída, será necessário implementar ou criar a base de dados no SGBD escolhido. Essa fase requer que o desenvolvedor conheça a Linguagem SQL e conheça o SGBD selecionado.

A importância de uma boa modelagem se deve ao fato de que as aplicações que estarão acessando a base de dados devem estar em consonância com o modelo desenvolvido. É muito desanimador e trabalhoso perceber a necessidade de alterar o modelo de dados, depois da base de dados programada e da aplicação do usuário desenvolvida. A verdade é que isso gera um retrabalho, uma vez que não só a implementação da base terá que ser refeita, mas também os diagramas e a aplicação deverão ser atualizadas na maioria dos casos.

A fase de modelagem é a principal etapa no desenvolvimento de uma base dados. Por isso é muito importante que se dedique tempo e esforço no desenvolvimento de uma boa modelagem da base de dados.

Modelo de Dados

O modelo da base de dados é também conhecido como *Esquema da base de dados*. Um esquema não deve mudar com frequência, porque uma vez alterado, tudo o que estiver envolvido com esse esquema (aplicações de usuários que acessem a base de dados, diagramas, etc.) terá de ser revisado e, muitas vezes, alterado.

O ideal é que haja *independência dos dados* em relação às aplicações que acessam aqueles dados. No entanto, essa independência é bastante difícil de alcançar, mas uma vez que alterada a estrutura (ou esquema) de uma base de dados, há a necessidade de se alterar também as aplicações que acessam aquela base. Por exemplo, imagine que depois que a base de dados foi implementada e a aplicação do usuário foi desenvolvida, é percebido que uma das tabelas da base de dados precise ser dividida em duas. A aplicação que acessava apenas a tabela X, agora tem que acessar as tabelas X e Y.

Consequentemente, a aplicação terá que ser alterada para refletir as mudanças que foram feitas no esquema da base de dados. Assim, a independência de dados lógica (alteração no esquema lógico de uma base de dados sem necessidade de alterar outros diagramas ou a aplicação) é muito difícil de ser alcançada e, portanto, deve ser evitada.

É claro que existem situações em que a alteração na base de dados pode ser feita sem que ocorram maiores transtornos. Por exemplo, precisa-se criar um índice na base de dados para aumentar o desempenho da mesma ou precisa-se alterar o tipo de dado de um atributo (por exemplo, alterar de *smallint* para *int*) isso não exigirá alterações nem nos diagramas, nem na aplicação. A esse tipo de independência denomina-se de **independência física dos dados**, onde é possível alterar o esquema físico da base de dados sem a necessidade de reescrever aplicações dos usuários ou alterar os modelos.

Cuidado!

Às vezes, mesmo mudanças simples, como a do exemplo acima, provocam grandes mudanças na aplicação. Por exemplo, o *bug* do milênio foi causado pelos tipos de dados utilizados. Em alguns casos mesmo só mudando os tipos de dados será necessário rever toda a aplicação.

Agora, que se explicou a importância de uma boa modelagem, vamos entender o que é um modelo.

Um modelo de dados compreende a descrição dos dados que devem ser armazenados pelo sistema e como esses dados devem se relacionar. Para que seja possível fazer essa descrição, é utilizada uma linguagem de modelagem, que pode ser textual ou gráfica. Um modelo de dados também deve explicitar os tipos dos dados armazenados e as restrições que esses dados possuem.

Os modelos de dados podem ser basicamente de dois tipos:

- **Modelo de dados conceitual**, que fornece uma visão mais próxima do modo como os usuários visualizam os dados realmente;
- **Modelo de dados lógico**, que fornece uma visão mais detalhada do modo como os dados estão realmente armazenados no computador.



É importante destacar que algumas literaturas tratam o modelo resultante do Modelo Lógico, que é dependente de um SGBD específico, de Modelo Físico.

Um ponto importante a se destacar aqui é que para ser possível criar o modelo da base de dados é necessário grande interação com o usuário ou o responsável pela análise de requisitos do sistema.

Essa interação se faz necessária uma vez que para o projetista da base de dados poder desenvolvê-la, ele deve ter uma clara compreensão do que o usuário espera do sistema, que tipos de relatórios o usuário espera que este disponibilize, bem como saber quais são os objetivos do sistema.

É importante destacar ainda que o primeiro passo antes de modelar e desenvolver um sistema é identificar as necessidades e definir os objetivos do projeto. Ou seja, deve estar claro para o desenvolvedor o que o cliente quer e o que ele espera do *software* que será desenvolvido.



Atividades

1) Dê três exemplos de situações, diferentes das citadas no capítulo 1, nas quais seria necessário utilizar uma base de dados.

2) Para cada uma das situações que você descreveu no exercício 1, descreva os dados que seriam úteis armazenar na base de dados.

3) O que significa dizer que as informações que vamos armazenar em uma base de dados devem ser úteis? Para quem elas devem ser úteis?

4) Defina o que caracteriza um Sistema de Banco de Dados computacional.

5) Qual a diferença entre uma base de dados e um Sistema de Banco de Dados?

6) Cite e explique as principais características de um SGBD.

7) O Access da Microsoft pode ser considerado um SGBD? Justifique sua resposta.

8) Com base no que foi trabalhado neste primeiro capítulo, explique qual a importância de uma boa modelagem de dados?

9) Faça uma síntese explicando o que é um modelo de dados e para que ele serve.

10) Vimos que é importante a participação do usuário na modelagem de uma base de dados. Apresente três argumentos a favor dessa participação.

11) Por que não é necessário que o usuário final tenha conhecimento de área de banco de dados?